Good Enough: Facing the Cost-Performance Tradeoff for LLMs in Computing Education*

Nicholas Gardella
Department of Computer Science
Washington and Lee University
Lexington, VA 24450
ngardella@wlu.edu

Abstract

Perspectives on LLMs in CS education (e.g. [3]) often rest on an assumption that human educators are more reliable curators of correct educational information for students than LLMs. This motivates restriction of LLM use for fielding student questions or providing grading feedback, for example. It also motivates the creation of deterministic software that can increase the reliability of LLMs, such as documentation lookup tools[1] that source reliable context to use as a basis for LLM generation. However, framing LLMs as generally unreliable has the downside to encourage *disuse*, the "neglect or underutilization of automation" [15]. Informational resources are typically not unlimited. Libraries hold finite copies of books, instructors have limited time, and computers incur costs to operate. As such, frugal use helps for all users (students in this case) to get their share. Framing LLMs as unreliable *should* discourage use and preserve resources. Instead, it can do quite the opposite. This proposal paper explains the LLM cost-performance tradeoff for computing educators using a practical example and a list of concrete recommendations for pedagogy, concluding with calls for needed future work.

*Copyright ©2026 by the Consortium for Computing Sciences in Colleges. Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the CCSC copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Consortium for Computing Sciences in Colleges. To copy otherwise, or to republish, requires a fee and/or specific permission.

[1]e.g. Context 7 (https://context7.com/)

1 Introduction

Large Language Models (LLMs) are well suited to computer programming and educational applications [17], but they are imperfectly reliable, creating potential harms for students [3, 18, 28, 32]. Computing students are increasingly using LLMs for educational help-seeking [7], so educators need protective yet realistic messaging on this topic. Though the perspectives of students and instructors differ [30], both parties show deep concerns about the real possibility for the LLMs they use to hallucinate and provide untrustworthy information [18]. This has motivated researchers to suggest educational integrations of LLMs that cope with their unreliability [3]. However, excessive focus on the unreliability of LLMs contributes to wasteful spending of the limited resources that enable them. This proposal paper challenges the unreliability narrative through the lens of the cost-performance tradeoff [2], promoting the use of less reliable models to promote economic and environmental stewardship [19]. After a realistic educational help-seeking example to illustrate this tradeoff, the primary contribution is a roadmap for how and why to adopt worse LLMs in computing education.

2 Background

2.1 Cost-Performance Tradeoff

LLMs compress insights about their training data into a non-deterministic prediction machine embodied as a collection of numerical weights and biases. All else being equal, larger LLMs are more reliable than smaller ones but are also more expensive—computationally, financially, and environmentally [2, 19]. This is because large LLMs need advanced computer hardware with high energy requirements to run effectively. Figure 1 shows the relative data size (i.e., the volume of 0s and 1s that need to fit into the computer hardware) for three modern LLMs (in 2026). The larger ones, Moonshot AI's Kimi K2.6[2] and K3[3], come in at 2.8 and 1.1 trillion parameters respectively, while the smaller, Microsoft's Phi 4 Mini[4], has just 4 billion parameters. All three models are open-weight, meaning anyone can freely download and run them on their own computer, but size can be practically prohibitive. Phi's parameters occupy a few gigabytes, requiring a modestly powerful consumer-grade computer to run, but the Kimi models require multiple terabytes, demanding high-performance clusters of industrial grade computers.

[2] `https://huggingface.co/moonshotai/Kimi-K2.6`
[3] `https://huggingface.co/moonshotai/Kimi-K3`
[4] `https://huggingface.co/microsoft/Phi-4-mini-instruct`

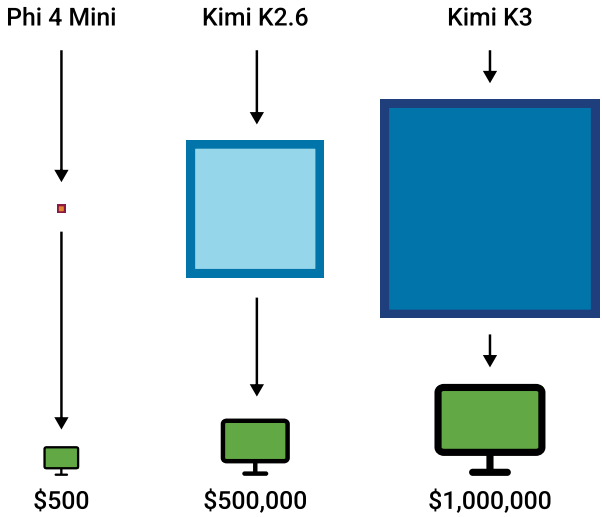


Figure 1: Scale diagram comparing a 4B LLM to larger 1.1T and 2.8T alternatives.

Note. Rectangle areas show, to scale, the relative data size of the numerical weights and biases that are the substance of an LLM. Reasonable estimates for the cost of the computing hardware required to run such models (in 2026 U.S. dollars) are provided. In reality, the Kimi models would not run on a single computer but rather many cooperating computers having the noted value in total.

2.2 Environmental and Social Motivations

Training and running LLMs is computationally intensive, so excessive reliability is wasteful. Computers incur substantial monetary cost and strain on the natural environment by using water and energy, much of which generates environmentally harmful carbon emissions [2, 19, 29]. Savings of money and natural resources cooperate nicely (e.g., saving energy saves money). Unfortunately, cost minimization can also encourage describing human workers in terms of their water, energy, and money usage compared to LLMs that can simulate their labor [19]. While out of scope here, critiquing this narrative within computing education is an important area for future work. What cost minimization gets right is that excess is wasteful. As the old idiom goes, *you ought not use a sledgehammer to crack a nut* [9]. Just the same, LLM users ought not use larger, more reliable LLMs if smaller, cheaper ones are just as reliable for the task at hand.

Smaller LLMs are also better for promoting equitable computing. Due to the costly computing needs of frontier LLMs (i.e. the largest, most reliable models of the day), control over these resources becomes centralized, even when model weights are open. LLM-based coding tools like GitHub Copilot, for example, offer users discounts to use an automatic model selector that prevents popular models from being overused [6]. By controlling the means of intellectual production, large Artificial Intelligence (AI) providers exert a centralized social control over individual programmers [23]. This pattern contrasts the democratic spirit of computer programming, which traditionally allowed individuals to build path-breaking technology with minimal capital resources (i.e., money and equipment). Centralized corporate control also threatens computing education because it creates financial barriers for students to participate in computing at the cutting edge [26]. If computing educators embrace small, less reliable LLMs, this not only prevents waste but also preserves the democratic, enterprising spirit of computing for students who want and need to use LLMs but cannot afford frontier options.

2.3 Existing Approaches

AI providers are using intelligent model routing to prevent waste in LLM inference. Though training LLMs accounts for a huge portion of their cost [14, 19], the present scope is focused on LLM inference, the stage where a given trained model is used repeatedly to service users' prompts. A core challenge with the cost-performance tradeoff is that LLMs are random by nature, so guessing how well they will perform is hard. However, AI researchers have developed intelligent strategies for routing and cascading users' prompts to prevent waste [2, 6, 14, 29]. For example, a model can be trained to guess which of a set of

increasingly reliable options will produce a response that is good enough for a given prompt [2]. As long as that prediction model is efficient, it saves cost by only using a more powerful model when needed. Another technique, cascading, involves a guess-and-check strategy, where an efficient response evaluator [31] can test the output of a small LLM, escalating to larger ones until the response is good enough [14]. Overall, there is great promise in automatically choosing LLMs that best balance reliability with cost.

Automatic model selection only works if more powerful LLMs are available when needed, so computing educators are exploring ways to meet their needs with only small LLMs. LLMs can simulate complex generation tasks that traditionally involve planning, reasoning, and multiple subtasks. More powerful LLMs can often do such tasks without additional tools, obviating the need to provide any. For example, a large frontier LLM might be able to retrieve a given fact solely out of its parameters, while a smaller LLM might be able to do so only if given a web search tool to use for context. As a result, some computing education researchers advocate for giving LLMs specified sub-tasks (e.g. summarizing compiler error messages or lists of search results) rather than broad objectives [3]. LLMs tend to perform better on narrower tasks, so small LLMs get huge reliability gains when used in this manner. For example, researchers have used multi-stage curation techniques with small LLMs to generate programming problems of similar quality to ones produced at baseline by large LLMs [8]. Given students are using LLMs for help seeking [7], other work has arranged small, specialized LLMs into personalized AI learning tools for students to approximate the performance of larger LLMs [25]. This promising area of work encourages resourceful stewardship of LLM technology while preserving the high reliability desired in education.

3 Educational Example

Next, the cost-performance tradeoff is examined through a case study of how two 2026 LLMs answer a fictional student question. In this scenario, Devin, a second-year university student, has come to office hours at the end of the first week of a second course in introductory computer science (CS2). Devin's CS1 course was taught in Python, but CS2 is taught in Java. Her question goes something like this (see Figure 1 for her attached code):

> In my Python course, I made a vehicle class where the VIN field was editable from outside the class code; can I do that in Java?

There are many ways for Devin to obtain an answer: (1) ask the instructor,

```python
from random import randint

class Vehicle:
    """a motor vehicle with a random vehicle identification number"""
    vin = None
    def __init__(self):
        """the valid VIN range is 75 to 95 (inclusive)"""
        min_vin, max_vin = 75, 95
        self.vin = randint(min_vin, max_vin)

honda = Vehicle()
print(honda.vin)
honda.vin = 80
print(honda.vin)
```

Figure 2: Code attachment to a fictional student question.

(2) research the question at a public library[5], (3a) input the question to a powerful, frontier LLM (e.g., Kimi 2.6; see Figure 1), and (3b) input the question to a basic, accessible LLM (e.g., Phi 4 Mini; see Figure 1). As explained in Cost-Performance Tradeoff, Kimi is more reliable than Phi, but Phi is cheaper.

Option (1), asking the instructor, could be nerve-wracking for the student, but would provide an effective and inviting social opportunity to resolve the concern [4, 5]. The library, option (2), ought to have the most reliable information on the topic, provided it carries two relevant texts on Python and Java programming. A social interaction there would feature the same tension of vulnerability and pleasantness as with the instructor, but the librarian is unlikely to be an expert on Object-Oriented Programming. Both LLMs would give low-stakes pseudo-social help, available into the late hours of the night (when the instructor is fast asleep and the library is closed).

Returning to the thesis from earlier, why does framing LLMs as unreliable encourage wasteful use of finite informational resources? The answer is simple: over-emphasizing unreliability biases students toward excessively intelligent LLMs. Suppose the university subsidizes Kimi and Phi for students to use. When LLMs are criticized for their imperfect reliability, and K3 is more reliable, why should the student ever choose Phi for their midnight query?

Morally, choosing Phi is more upstanding if it *can* answer the question, as it would do so more frugally, preserving Kimi for others who really need it (see

[5]Note: internet search was excluded, as search engines are increasingly using LLM-based features

Background). This particular student question is useful because it does not necessarily have an exact right answer. Here is how one computing educator *believes* this question should be answered.

> Technically, yes, you can label a field as public to make it readable and writable outside the class definition as a property. However, in Java and OOP culture, this is strongly discouraged, because it makes the vehicle class harder to modify later without breaking code that uses it. In fact, your question is a good opportunity to explore how to make your Python code more maintainable by restricting what users of the vehicle class can do to your internal vin field. I can show you code examples for how to make a public field in Java (discouraged), make a private field with get and set methods, and update your Python code to use this approach as well.

Phi ran on a $500 gaming computer in an author's home, and started its answer as follows.

> In Java, you cannot directly modify a private field from outside the class like in Python...

Was it correct? No, its statement is technically false. However, this is not very harmful, since the falsity tells the student that a bad practice is actually impossible. At the time of writing, Kimi K3 was unavailable due to high demand, but K2.6 ran on Moonshot's servers somewhere far away, and began its answer as follows.

> Yes, you can do that in Java, but **you shouldn't**...

It gave a reliable response, but was it worth the astronomically higher cost? Phi can be coerced[6] to offer the simple correct line: "public int vin;" but it did not throw this out easily in its first explanation. It is very unlikely that the student here could produce the proper follow-up questions to make this happen. Similarly, a simple system prompt[7] can coerce Phi to a more correct answer, but the student may not think to do this, either. Thus, a more reliable system than Phi alone is needed to answer student questions like this one. How much more reliable does it really need to be, though? Maybe it is just eight or 12 billion parameters that will usually satisfy one's standards for answering

[6]Prompt #2: "But it is technically possible, right? I could make the vin field so that it can be directly modified with no getters and setters, right? But it is bad practice?"

[7]System Prompt: "encourage best practices but always acknowledge when bad practices are technically possible and show how"

this question. Instead, maybe a better harness for Phi with a different system prompt can garner the performance needed. Even then, the answer may be incorrect or incomplete, just as the human-written answer may seem to any given reader. Yet, perhaps Phi's answer is "good enough." How consequential is this error, and how different is it from if a human educator provided the same incorrect answer?

Overall, Phi is reliable for producing correct code and useful answers to questions. There are options more costly and reliable than Phi, but less costly and reliable than K2.6. The tradeoff always scales; at some point, an answer has to be "good enough" so to not let the perfect be the enemy of the good [2].

4 Recommendations for Educators

The example above illustrates how a small LLM may genuinely be less reliable than a larger one, causing student interactions to propagate misinformation. This next section describes several possible reactions to this fact and comments on their relative merits in light of the overall thesis that computing educators should not frame LLMs as unreliable. The reactions include abandoning, accepting, and coping with unreliability.

4.1 Abandoning Unreliable LLMs

The first reaction to the Educational Example above is to conclude that small LLMs are not reliable enough for use in education and should therefore be abandoned in favor of larger, costlier, and more reliable alternatives. A key basis for this proposal paper is the conjecture that as long as individuals are not personally responsible for the costs incurred, they will always seek out the smartest models readily available to them. Evaluating the validity of this conjecture is an important avenue for future work but is out of scope here. The challenge with abandoning small models in favor of larger ones is that the reliability of a given model is highly contextual. For the example above, with the given models and some particular evaluation criteria, Phi failed. That Phi can be coerced to produce a reliable response shows that there is promise to expose a better answer than the naïve approach gave. As such, going to a more expensive model is almost surely wasteful to some extent. That being said, perhaps a naïve approach can still be taken (with less effort for the user) with a slightly larger model (e.g., 8B parameters) without always using a frontier model for every task. For this, approaches such as automatic model selection are very promising [2, 6, 14]. Another approach is for instructional staff to manually sort student help-seeking requests, grading needs, or problem generation tasks based on their apparent complexity, sending simpler ones to lower cost LLMs and complex ones to more performant LLMs.

4.2 Accepting Unreliable LLMs

The next reaction to Phi's unreliability is to allow it knowingly. Assuming that an LLM will be often wrong as opposed to rarely wrong will change the risk associated with different modes of use. For example, [3] proposes that LLMs can be used as simulated students or instructor supports without creating direct risks for students when they give incorrect outputs. They argue, using Vygotsky's theory of social constructivism [27], that educational media (e.g., an AI-generated programming example) need not be perfectly correct as long as reliable human instructors can reconcile the media with correct knowledge structures. This approach is also consistent with learning research more generally, where students' incorrect answers to open-ended questions do not harm their learning provided instructors follow-up with correct answers later [11]. While not ideal, accepting the unreliability of small LLMs still provides educational utility without the waste associated with larger models.

4.3 Coping with Unreliable LLMs

The final reaction to Phi's unreliability is to cope with it better. The example notes how follow up prompts and system prompts can change the reliability outcome. This concept extends more formally into the resourceful application of small LLMs introduced in Existing Approaches. Coping strategies are the primary path recommended in this proposal, and there are two large approaches to consider. These include context customizations at training time and context adjustments at inference time.

Training-time customizations are less actionable for educators but potentially more powerful. In this category, educators can train or fine-tune LLMs specifically for their needs. For example, in [22], researchers took existing small LLMs, and used fine-tuning to increase their reliability at explaining compiler error messages. A downside of this process is that it wastes the considerable resources associated with training, since one LLM cannot be reused for many different purposes [19, 22].

4.3.1 Context Files

A more actionable (and sustainable) approach for most educators is inference-time customization via "system-level" context files. System prompts are additional text included with each user prompt (e.g., a student question) to steer how the LLM should respond. AI-driven programming tools (e.g., GitHub Copilot, Cursor, Claude Code, Pi Coding Agent, and OpenCode), accept several forms of system prompting, even when powered by small, offline LLMs. Being text based, system prompting is typically materialized as AI-directed context documents within the filesystem folder for a given programming project.

If a single git repository were used to store the code and materials for a programming course, then these additional files could be authored by the course instructor to improve how students' LLMs behave when offering them help for the class (so long as their AI tools have access to the folder). This process helps to combat the abandonment problem with guardrailed educational AI, where students simply use unprotected options like ChatGPT in addition to an instructor-provided LLM [10]. The additional context can make the instructor-provided LLM perform better than a frontier LLM at baseline, making abandonment less attractive. The most generic way to provide this context is as text files in any format. A reasonable convention might be a folder at the path .agents/context containing several markdown files such as syllabus.md, final-project.md, sample-exam.md, teaching-unit-testing-guidelines.md, etc., each with appropriate details. Apt naming and high granularity are best, as they help for irrelevant context to be ignored when needed.

Beyond improving LLM reliability, instructors can find new pedagogical opportunities through system prompting. Educators often have many topics they wish for students to learn, but students can become overwhelmed from too much information [24]. Some evidence suggests that covering less course material in greater depth is better for students' long-term retention than covering more material [20, 21]. By embedding a larger breadth of information to an LLM within context files, the instructor can deliver this information opportunistically based on student prompts, without overloading the general course schedule.

4.3.2 Rules Files

A more standard system prompting format is the instructions or rules file. This type of file describes how the LLM should and should not respond. The common format is an AGENTS.md file at the root of a project, but other harness-specific (e.g. Copilot v. Pi) naming conventions are also used. Because rules are typically used for all prompts, they should be general, so week-2-oop-inheritance.md would be poorly suited for a rules file. For rules, the instructor can specify relevant details about the course such as the primary language, the expertise level of the students, and to what extent answers to students should contain code snippets [12, 13].

4.3.3 Skills Files

A second canonical system prompting format is the *skill* file. AI *Agent Skills* are a standardized[8] way to describe specialized instructions for certain AI behaviors. Skills are indexed based on a short description; LLMs can then identify

[8] `https://agentskills.io/`

which of its available skills to apply in a given scenario. A challenge with simply placing all details in rules files (or a single overarching skill) is that the LLM's context window, a technical limitation for how much text it can consider for its response, can be filled up by irrelevant details. Since this window is shared with the user prompt itself, instructors should be cautious not to overfill the context, leaving little remaining for student-provided context (e.g., the code they are struggling to debug). Skills should therefore be short, concise patterns that then branch out to other files when needed. As of July 2026, Matt Pocock [16] is one of the most prominent and prolific authors of AI skills on Vercel's Skills sharing website[9]. At the time of writing, his "teach" skill[10] was the 89th most popular skill on the site, with nearly 350,000 downloads. The text for the skill contains many elements from research information theory and pedagogy such as Ackoff's Data-Information-Knowledge-Wisdom hierarchy [1] and Vygotsky's Zone of Proximal Development [27]. However, further evidence-informed skill formalization from computing education researchers is a major future work need.

4.3.4 File References

To keep rules, skills, and other context files concise, educators should make heavy use of file references (i.e., links). References are essential for providing system-level context to small LLMs because they help to preserve context windows, which are limited and degrade reliability as they fill. Using rules files and lists of skills as the root, an LLM can automatically detect relevant skills to use and appropriate links to follow. By placing an explanation about how to approach the teaching of arrays in a file referenced from a rules file, for example, a student asking about strings would not have their LLM's context window wasted by unnecessary details about arrays.

4.3.5 Tools

Tools allow LLMs (with tool-calling compatibility) to take actions on a user's behalf. Tools can be referenced in context files by standard names (e.g., "edit," or "Bash") or by path (e.g., ".agents/scripts/format-code.lua"). Useful tools for LLMs in education include code linters, compilers, and debuggers, all of which are deterministic avenues to help with students' needs (e.g., "why is my program crashing?") [22]. Tools are especially important for small LLMs because they allow highly reliable deterministic software to underly responses that might otherwise be unreliable.

[9] `https://www.skills.sh/`
[10] `https://www.skills.sh/mattpocock/skills/teach`

4.4 Recommendation Summary

Educators' concerns about unreliable LLMs are valid, but there still many ways they can protect their students without defaulting to expensive frontier offerings. First, those unwilling to accept or cope with unreliability can step up to the cheapest models that meet their needs. Second, those who cannot tolerate unreliability at all can use LLMs as untrustworthy peers or generators of media for critique. Finally, the best approach is to use context files and tools for inference-time customization that improves overall reliability. These recommendations provide a useful starting point for educators looking to introduce LLMs to their students with an eye towards equity, sustainability, and effective learning. However, there is a considerable need to formalize these recommendations into complete guidelines and ready-to-use context packages for various computing courses. Another important area of future work is ethical student monitoring. Platforms like CodeHelp [12] can help computing educators monitor student queries, but additional work is needed to translate this data into pedagogical countermeasures to AI misinformation.

5 Conclusion

In conclusion, this proposal paper has explored a case study of the cost-performance tradeoff inherent to using Large Language Models in computing education. It walks through a relevant example of a student query and the real responses of two 2026 LLMs at opposite ends of the cost spectrum. This provides a starting point for computing educators to gain comfort with the imperfect reliability of LLMs and begin using them with frugality and stewardship. Future work should continue to explore methods for calibrating the tradeoff between cost and reliability in different computing courses [8]. Technical problems like intelligent routing of student queries to differentially sized models should also be explored further [2, 14, 25]. Finally, the computing education community desperately needs better educational guidelines and materials for using modern AI coding tools with small, unreliable LLMs.

References

[1] Russell L. Ackoff. "From data to wisdom". In: *Journal of applied systems analysis* 16.1 (1989), pp. 3–9. URL: `http://faculty.ung.edu/kmelton/Documents/DataWisdom.pdf` (visited on 07/30/2026).

[2] Dujian Ding et al. “Hybrid llm: Cost-efficient and quality-aware query routing”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=02f3mUtqnM (visited on 07/30/2026).

[3] Nickolas Falkner, Leo Leppänen, and Juho Leinonen. “Effective Use of Large Language Models for Social Constructivism in Computer Science Education”. In: *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education V. 1*. ITiCSE 2026. New York, NY, USA: Association for Computing Machinery, July 2026, pp. 210–216. ISBN: 979-8-4007-2634-7. DOI: 10.1145/3803400.3809392. URL: https://dl.acm.org/doi/10.1145/3803400.3809392 (visited on 07/24/2026).

[4] Guangrui Fan et al. “The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance: a comparative study with traditional pair programming and individual approaches”. en. In: *International Journal of STEM Education* 12.1 (Mar. 2025), p. 16. ISSN: 2196-7822. DOI: 10.1186/s40594-025-00537-3. URL: https://stemeducationjournal.springeropen.com/articles/10.1186/s40594-025-00537-3 (visited on 02/09/2026).

[5] Nicholas Gardella et al. “HBCU Student Perspectives on Identity, Persistence, and Code-Generating AI in CS Education: A Case Study”. en. In: *Proceedings of the 25th Koli Calling International Conference on Computing Education Research*. Koli Finland: ACM, Nov. 2025, pp. 1–12. ISBN: 979-8-4007-1599-0. DOI: 10.1145/3769994.3770015. URL: https://dl.acm.org/doi/10.1145/3769994.3770015 (visited on 02/09/2026).

[6] GitHub. *About Copilot auto model selection*. en. 2026. URL: https://docs-internal.github.com/en/copilot/concepts/models/auto-model-selection (visited on 07/30/2026).

[7] Irene Hou et al. “The Effects of Generative AI on Computing Students’ Help-Seeking Preferences”. In: *Proceedings of the 26th Australasian Computing Education Conference*. ACE ’24. New York, NY, USA: Association for Computing Machinery, Jan. 2024, pp. 39–48. ISBN: 979-8-4007-1619-5. DOI: 10.1145/3636243.3636248. URL: https://dl.acm.org/doi/10.1145/3636243.3636248 (visited on 06/23/2025).

[8] Oscar Jimenez Rodriguez. “Personalized Coding Problem Generation Using Open-source Small Language Models”. In: *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.2*. Vol. 2.

SIGCSE TS 2026. New York, NY, USA: Association for Computing Machinery, Feb. 2026, pp. 1667–1668. ISBN: 979-8-4007-2255-4. DOI: 10.1145/3770761.3777153. URL: https://dl.acm.org/doi/10.1145/3770761.3777153 (visited on 07/30/2026).

[9] Levi Carroll Judson. *The Sages and Heroes of the American Revolution*. Literary Licensing, LLC, 1875. ISBN: 978-1-4981-0037-3. (Visited on 07/31/2026).

[10] Majeed Kazemitabaar et al. "CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs". In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. CHI '24. New York, NY, USA: Association for Computing Machinery, May 2024, pp. 1–20. ISBN: 979-8-4007-0330-0. DOI: 10.1145/3613904.3642773. URL: https://dl.acm.org/doi/10.1145/3613904.3642773 (visited on 06/23/2025).

[11] James M. Lang. *Small teaching: Everyday lessons from the science of learning*. John Wiley & Sons, 2021. URL: https://books.google.com/books?hl=en&lr=&id=k8E4EAAAQBAJ&oi=fnd&pg=PR1&dq=small+teaching&ots=i3Us4IP_Jf&sig=tbrpYuHb8my96pQu2z3uAHWapwY (visited on 07/24/2026).

[12] Mark Liffiton et al. "CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes". en. In: *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. Koli Finland: ACM, Nov. 2023, pp. 1–11. ISBN: 979-8-4007-1653-9. DOI: 10.1145/3631802.3631830. URL: https://dl.acm.org/doi/10.1145/3631802.3631830 (visited on 06/23/2025).

[13] Rongxin Liu et al. "Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education". In: *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. SIGCSE 2024. New York, NY, USA: Association for Computing Machinery, Mar. 2024, pp. 750–756. ISBN: 979-8-4007-0423-9. DOI: 10.1145/3626252.3630938. URL: https://dl.acm.org/doi/10.1145/3626252.3630938 (visited on 06/23/2025).

[14] Yasmin Moslem and John D. Kelleher. "Dynamic model routing and cascading for efficient LLM inference: A survey". In: *arXiv preprint arXiv:2603.04445* (2026). URL: https://openreview.net/pdf?id=KqE3WofJ6C (visited on 07/30/2026).

[15] Raja Parasuraman and Victor Riley. "Humans and Automation: Use, Misuse, Disuse, Abuse". en. In: *Human Factors* 39.2 (June 1997), pp. 230–253. ISSN: 0018-7208. DOI: 10.1518/001872097778543886. URL: https://doi.org/10.1518/001872097778543886 (visited on 10/11/2024).

[16] Matt Pocock. *mattpocock/skills*. original-date: 2026-02-03T11:15:53Z. July 2026. URL: https://github.com/mattpocock/skills (visited on 07/30/2026).

[17] James Prather et al. "Beyond the Hype: A Comprehensive Review of Current Trends in Generative AI Research, Teaching Practices, and Tools". In: *2024 Working Group Reports on Innovation and Technology in Computer Science Education*. ITiCSE 2024. New York, NY, USA: Association for Computing Machinery, 2025, pp. 300–338. ISBN: 979-8-4007-1208-1. DOI: 10.1145/3689187.3709614. URL: https://doi.org/10.1145/3689187.3709614.

[18] James Prather et al. "The Robots Are Here: Navigating the Generative AI Revolution in Computing Education". In: *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*. ITiCSE-WGR '23. New York, NY, USA: Association for Computing Machinery, Dec. 2023, pp. 108–159. ISBN: 979-8-4007-0405-5. DOI: 10.1145/3623762.3633499. URL: https://dl.acm.org/doi/10.1145/3623762.3633499 (visited on 09/03/2024).

[19] Shaolei Ren et al. "Reconciling the contrasting narratives on the environmental impact of large language models". en. In: *Scientific Reports* 14.1 (Nov. 2024), p. 26310. ISSN: 2045-2322. DOI: 10.1038/s41598-024-76682-6. URL: https://www.nature.com/articles/s41598-024-76682-6 (visited on 07/30/2026).

[20] Brendan A. Schuetze and Veronica X. Yan. "Optimal Learning Under Time Constraints: Empirical and Simulated Trade-offs Between Depth and Breadth of Study". en. In: *Cognitive Science* 46.4 (Apr. 2022), e13136. ISSN: 0364-0213, 1551-6709. DOI: 10.1111/cogs.13136. URL: https://onlinelibrary.wiley.com/doi/10.1111/cogs.13136 (visited on 02/11/2026).

[21] Marc S. Schwartz et al. "Depth versus breadth: How content coverage in high school science courses relates to later success in college science coursework". en. In: *Science Education* 93.5 (Sept. 2009), pp. 798–826. ISSN: 0036-8326, 1098-237X. DOI: 10.1002/sce.20328. URL: https://onlinelibrary.wiley.com/doi/10.1002/sce.20328 (visited on 07/30/2026).

[22] Lorenzo Lee Solano et al. "Fine-Tuning Open-Source Models as a Viable Alternative to Proprietary LLMs for Explaining Compiler Messages". In: (2026). URL: https://aaltodoc.aalto.fi/items/0cfcbee8-ebe8-4d3b-a9a2-2f3714e5fe2f (visited on 07/30/2026).

[23] Ali Sunyaev, Michel Avital, and Mary C. Lacity. "From ideology to design: Toward purposeful decentralization of information systems governance". en. In: *Journal of Information Technology* 41.1 (Mar. 2026), pp. 2–14. ISSN: 0268-3962, 1466-4437. DOI: 10.1177/02683962261430919. URL: https://journals.sagepub.com/doi/10.1177/02683962261430919 (visited on 07/30/2026).

[24] John Sweller. "Cognitive load theory". In: *Psychology of learning and motivation*. Vol. 55. Elsevier, 2011, pp. 37–76. URL: https://www.sciencedirect.com/science/article/pii/B9780123876911000028 (visited on 12/28/2023).

[25] Shilpi Taneja et al. "Agentic RAG for Personalized Learning: Design of an AI-Powered Learning Agent Using Open-Source Small Language Models". en. In: *Electronic Journal of e-Learning* 23.4 (Nov. 2025), pp. 69–80. ISSN: 1479-4403. DOI: 10.34190/ejel.23.4.4044. URL: https://academic-publishing.org/index.php/ejel/article/view/4044 (visited on 07/30/2026).

[26] University of New South Wales et al. "Exploring the Intersection of the Digital Divide and Artificial Intelligence: A Hermeneutic Literature Review". In: *AIS Transactions on Human-Computer Interaction* 12.4 (Dec. 2020), pp. 253–275. ISSN: 19443900. DOI: 10.17705/1thci.00138. URL: https://aisel.aisnet.org/thci/vol12/iss4/5 (visited on 07/30/2026).

[27] Lev S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Ed. by Michael Cole et al. Cambridge, MA: Harvard University Press, 1978. ISBN: 978-0-674-57629-2.

[28] Shen Wang et al. "Large Language Models for Education: *A survey and outlook*". In: *IEEE Signal Processing Magazine* 42.6 (Nov. 2025), pp. 51–63. ISSN: 1053-5888, 1558-0792. DOI: 10.1109/MSP.2025.3594309. URL: https://ieeexplore.ieee.org/document/11364183/ (visited on 07/30/2026).

[29] Jasmine Wu and Deirdre Bosa. "Model routing on AI is a problem for OpenAI and Anthropic". In: *Consumer News and Business Channel* (). URL: https://www.cnbc.com/2026/06/05/model-routing-on-ai-is-a-problem-for-openai-and-anthropic.html (visited on 07/30/2026).

[30] Cynthia Zastudil et al. "Generative AI in Computing Education: Perspectives of Students and Instructors". In: *2023 IEEE Frontiers in Education Conference (FIE)*. Oct. 2023, pp. 1–9. DOI: 10.1109/FIE58773.2023.10343467. URL: https://ieeexplore.ieee.org/document/10343467 (visited on 06/23/2025).

[31] Tianyi Zhang et al. *BERTScore: Evaluating Text Generation with BERT*. arXiv:1904.09675 [cs.CL]. Feb. 2020. DOI: 10.48550/arXiv.1904.09675. URL: http://arxiv.org/abs/1904.09675 (visited on 07/31/2026).

[32] Kyrie Zhixuan Zhou et al. *"The teachers are confused as well": A Multiple-Stakeholder Ethics Discussion on Large Language Models in Computing Education*. arXiv:2401.12453 [cs.CY]. Jan. 2024. DOI: 10.48550/arXiv.2401.12453. URL: http://arxiv.org/abs/2401.12453 (visited on 07/30/2026).